How Runtime Enforcement Works

We assume that the policy violations we aim to prevent occur at tool-call boundaries, including message sends. Our tool instruments these entry points to record policy-relevant information and check each new request against rules and current facts—for example, whether the caller may execute a transfer. Calls that fail the checks are blocked before the tool body runs.

This includes MCP tool calls and A2A message sends when their call boundaries are instrumented. The app supplies who is calling, the arguments and the facts needed by the rules. Enforcement covers calls that pass through those instrumented entry points.

1 Builtins and their semantics

A **fact** is a recorded piece of information. For example, `Parameter("q1", "recipient", "Bob")` records that request `q1` has a parameter named `recipient` whose value is `Bob`. In the general form `Parameter(Q, Name, Value)`, `Q` identifies the request, `Name` names a parameter, and `Value` gives its value. Our tool stores facts in a fact database (EDB).

A **predicate** names a kind of fact, such as `Parameter`. A **rule** tells us what follows from facts we already have. Read `:-` as "if" and commas on its right as "and": the conclusion on the left holds when all conditions on the right hold. Letters such as `P` and `R` stand for values; the same letter within a rule refers to the same value.

Runtime gate

`Request(Q, Principal, EventKind)`: Request `Q` is made by `Principal` to perform `EventKind`.

`Parameter(Q, Name, Value)`: Request `Q` has this parameter and value.

`Allow(Q)`: The rules permit request `Q`.

`Deny(Q, Reason)`: The rules reject request `Q` for this reason.

Allow(Q)	Any Deny(Q, Reason)	Tool body
Absent	Absent	Blocked
Present	Absent	Allowed to run
Absent	Present	Blocked
Present	Present	Blocked

Our tool checks each request using the current facts. A request needs `Allow`; any `Deny` blocks it.

Category 1: Capability / access control

`Capability(Principal, Scope, EventKind)`: This principal has permission to perform this operation on this resource or group of resources.

`PrincipalGroup(Principal, Group)`: This principal belongs to this group.

`ResourceGroup(Parent, Child)`: This resource belongs to a larger resource group.

`Can(Principal, Resource, EventKind)`: This principal has the permission, directly or through a group.

```
Can(P, R, E) :- Capability(P, R, E).
Can(P, R, E) :- PrincipalGroup(P, G), Can(G, R, E).
Can(P, Child, E) :- ResourceGroup(Parent, Child), Can(P, Parent, E).
```

The first rule reads: a principal can perform an operation on a resource if it has that capability. The next two rules extend permissions through principal and resource groups. The app's `Allow` rules use `Can` to check permission.

Category 2: Information flow control

`Source(Principal, Data)`: This data came directly from this principal.

`DependsOn(Output, Input)`: This output was produced using this input.

`Provenance(Data, Principal)`: This data came from this principal, directly or through other data.

```
Provenance(D, P) :- Source(P, D).
Provenance(Out, P) :- DependsOn(Out, In), Provenance(In, P).
```

The app records where data comes from and which inputs it uses. These rules carry the origins forward when data is copied, combined or rewritten.

Example 1. Policy: Only Payments may execute transfers. Payment details from external messages require additional verification.

The transfer rule checks the caller's `Can` permission and uses `Provenance` to require verification for external-message details. Rewriting those details keeps their recorded origin, so it cannot bypass the check.

Category 3: State / lifecycle

`ResourceType(Resource, Type)`: What kind of resource this is.

`ResourceState(Resource, State)`: What state the resource is in now.

`InitialState(Type, State)`: The starting state for a new resource of this type.

`AllowedTransition(Type, EventKind, From, To)`: This operation may move this type of resource from one state to another.

`ReachableState(Type, From, To)`: One or more allowed steps connect these states.

`CanTransition(Resource, EventKind, From, To)`: This resource can take this step from its current state.

```
ReachableState(T, From, To) :- AllowedTransition(T, E, From, To).
ReachableState(T, From, To) :- ReachableState(T, From, Mid),
                               AllowedTransition(T, E, Mid, To).
CanTransition(R, E, From, To) :- ResourceType(R, T),
    ResourceState(R, From), AllowedTransition(T, E, From, To).
```

These rules connect the allowed steps and check which step is available now. The app's `Allow` rules use `CanTransition` to check the next step; our tool updates `ResourceState` when the operation succeeds.

Example 2. Policy: A document may be published only after approval.

Configure `DRAFT` → `APPROVED` → `PUBLISHED`. A publish rule requires the `APPROVED` → `PUBLISHED` transition: publishing a draft is blocked; successful publication updates its state.

2 What changes in your app

Our tool adds three pieces:

1. **A `.dl` file** containing application rules and base facts, such as who has permission and which state changes are allowed.
2. **`@policy` on tool functions.** Before an annotated function runs, our tool records `Request` and `Parameter`, checks the rules, and blocks calls that are not permitted.
3. **Fact writes inside tool functions.** Added statements record known application information and results in the EDB for later checks.

Grey lines show the additions to a message-send function:

```
@policy
def send_message(recipient, body):
   result = transport.send(recipient, body)
   policy.put("Sent", result.id, recipient)
   return result
```

`@policy` checks the caller and recipient before sending. After a successful send, `policy.put` records which message was sent to which recipient. Later requests can be checked against that fact.